

Technical Report

Final Assignment

CMP4272



Priyangan Vasantharajan

Student ID: 25197298

BSc(Hons) Computer Science with Ai



Birmingham City University

A1

A1.1 – Search by Name

The `search_by_name()` method searches the tree for students whose name matches the entered name. The method returns all matching student records.

Testing Proof:

```
===== Student Records Menu =====
1. Display
2. Insert Student Record
3. Search for a Student by ID
4. Search for Students by Name
5. Display Total Student Count - Recursive
6. Display All Student Records (Sorted by ID)
7. Display Total Student Count - Iterative
8. Update student record
-1. Exit

Enter your choice: 1
=====Display all data=====
ID: 101, Name: Ethan Hunt, marks: 3.2
ID: 102, Name: Bob Smith, marks: 3.8
ID: 103, Name: Kyle Johnson, marks: 3.3
ID: 104, Name: Hannah Lee, marks: 3.6
ID: 105, Name: Alice Johnson, marks: 3.5
ID: 106, Name: Liam Scott, marks: 3.75
ID: 107, Name: Diana Prince, marks: 3.7
ID: 108, Name: George Miller, marks: 3.1
ID: 109, Name: Jessica Davis, marks: 3.0
ID: 110, Name: Charlie Brown, marks: 2.9
ID: 112, Name: Ian Wright, marks: 3.4
ID: 115, Name: Fiona Clark, marks: 3.9
```

A1.2 – Total Student Count (Recursive)

The `total_student_count()` method is used to count the total number of students stored in the Binary Search Tree. It uses a recursive approach to visit the nodes in the tree and count each student record. The final count is returned after all the student nodes have been visited.

Testing:

The method was tested using option 5 from the main menu. The output returned a total of **14 students**, confirming that the recursive counting method worked successfully.

Testing Proof:

```
Enter your choice: 5
Total number of students: 14 - recursive
```

A1.3 – Display Ordered Student Records

The `display_ordered()` method is used to display all student records in ascending order based on the student ID. It arranges the records by ID and displays the student ID, name and marks in the correct order.

Testing:

The method was tested using option 6 from the main menu. The output shows that all student records were successfully displayed in ascending order of student ID.

Testing Proof:

```
Enter your choice: 6

Student Records (Sorted by Id ):
ID:101, Name:Ethan Hunt, Marks:3.2
ID:101, Name:Ethan Hunt, Marks:3.2
ID:102, Name:Bob Smith, Marks:3.8
ID:103, Name:Kyle Johnson, Marks:3.3
ID:104, Name:Hannah Lee, Marks:3.6
ID:105, Name:Alice Johnson, Marks:3.5
ID:106, Name:Liam Scott, Marks:3.75
ID:107, Name:Diana Prince, Marks:3.7
ID:108, Name:George Miller, Marks:3.1
ID:109, Name:Jessica Davis, Marks:3.0
ID:110, Name:Charlie Brown, Marks:2.9
ID:112, Name:Ian Wright, Marks:3.4
ID:115, Name:Fiona Clark, Marks:3.9
ID:1111, Name:saran, Marks:99.0
```

A1.4 – Total Student Count (Iterative)

The `total_student_count_iter()` method is used to count the total number of students in the Binary Search Tree using an iterative approach. It uses a stack to visit each node in the tree and increases the count for every student record found.

Testing:

The method was tested using option 7 from the main menu. The output returned a total of **14 students**, showing that the iterative counting method worked correctly.

```
Enter your choice: 7
Total number of students: 14 - Iterative
```

A1.5 – Update Student Record

The `update_student_record()` method is used to update an existing student record using the student ID. First, the method searches for the given student ID. If the student is found, the user can choose to update either the student's name or marks. If the student ID does not exist, the system displays a "Student not found" message.

Testing:

The method was tested using option 8 from the main menu. Student ID **115** was selected, and both the name and marks were tested separately. The successful messages confirmed that the student record was updated correctly.

Testing Proof:

```
Enter your choice: 8
  Enter ID :115
1.Name
2.Marks
Enter Your Choice Number:1
Enter the Name:Hari
Student record updated successfully

===== Student Records Menu =====
1. Display
2. Insert Student Record
3. Search for a Student by ID
4. Search for Students by Name
5. Display Total Student Count - Recursive
6. Display All Student Records (Sorted by ID)
7. Display Total Student Count - Iterative
8. Update student record
-1. Exit
Enter your choice: 8
  Enter ID :115
1.Name
2.Marks
Enter Your Choice Number:2
Enter the Marks:55
Student record updated successfully
```

Activate Wi
Go to Settings t

A2

A2.1 – Tree: Insert Node

The insert() method is used to add a new student record into the Binary Search Tree. First, a new node is created using the student ID, name and marks. If the tree is empty, the new student becomes the root node. If the tree already contains student records, the new student ID is compared with the current student ID.

If the new ID is smaller, the method moves to the left side of the tree. Otherwise, it moves to the right side. This process continues until an empty position is found, where the new student is inserted. This keeps the student records organised according to their student IDs.

```
10
11     def insert(self, student_id, name, marks):
12         """Insert a student record into the tree """
13         new_node = StudentNode(student_id, name, marks)
14         if self.root is None:
15             self.root = new_node
16             return
17
18         current = self.root
19         while True:
20             if student_id < current.student_id:
21                 if current.left is None:
22                     current.left = new_node
23                     return
24                 current = current.left
25             else:
26                 if current.right is None:
27                     current.right = new_node
28                     return
29                 current = current.right
30
```

A2.2 – Graph: Create Edge

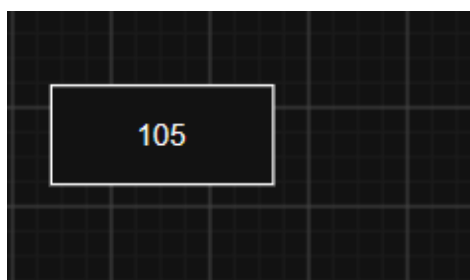
The `add_friendship()` method is used to create a friendship connection between two students in the graph. First, the method checks whether both students are already available in the graph. If they are not available, they are added first. After that, each student is added to the other student's friendship list. This creates a two-way connection between the students. Therefore, if Student A is connected to Student B, Student B is also connected to Student A.

```
10 def add_friendship(self, student1, student2):
11     """Create an undirected edge (friendship)
12     between two students."""
13     if student1 not in self.graph:
14         self.add_student(student1)
15     if student2 not in self.graph:
16         self.add_student(student2)
17
18     self.graph[student1].append(student2)
19     self.graph[student2].append(student1)
```

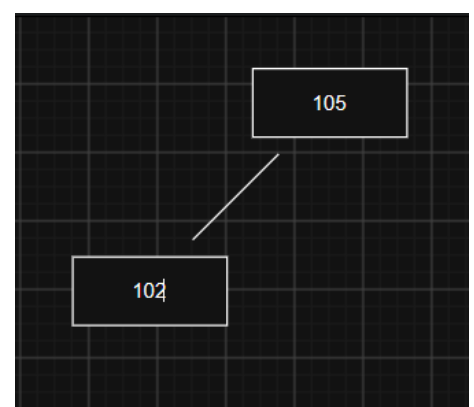
A3.1 – BST Creation Using the Original Insertion Order

The sequence shows how each student ID is inserted into the BST based on its value. Smaller IDs are placed on the left and larger IDs are placed on the right until all 12 student records are inserted.

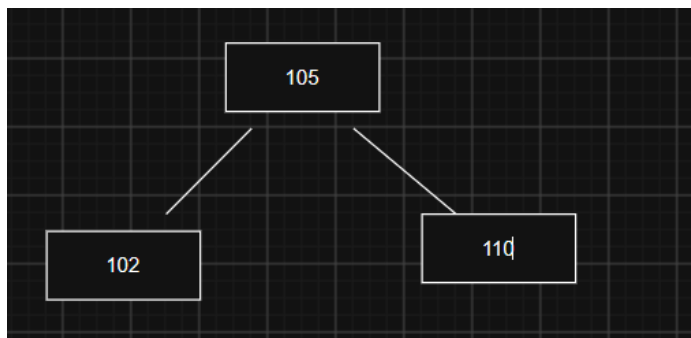
1



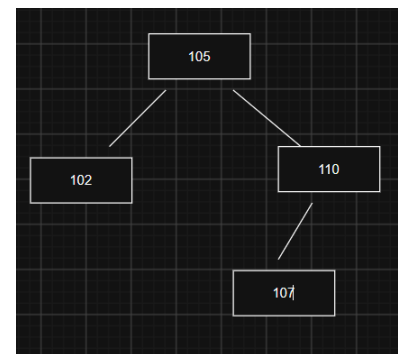
2



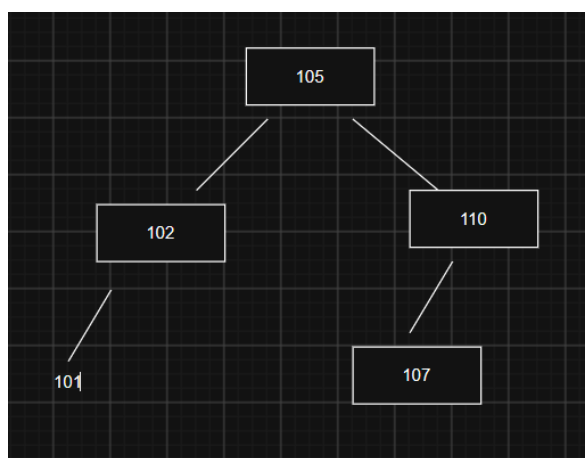
3



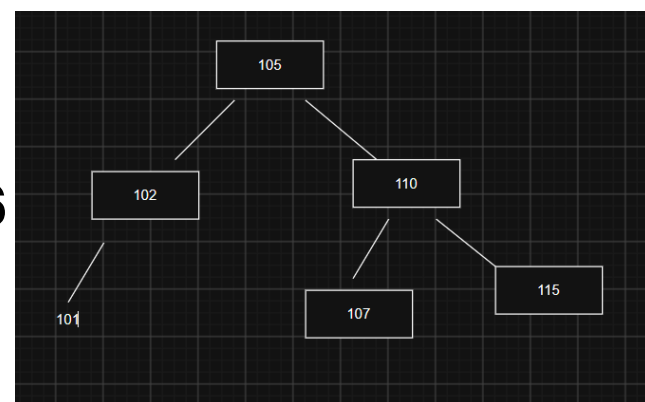
4



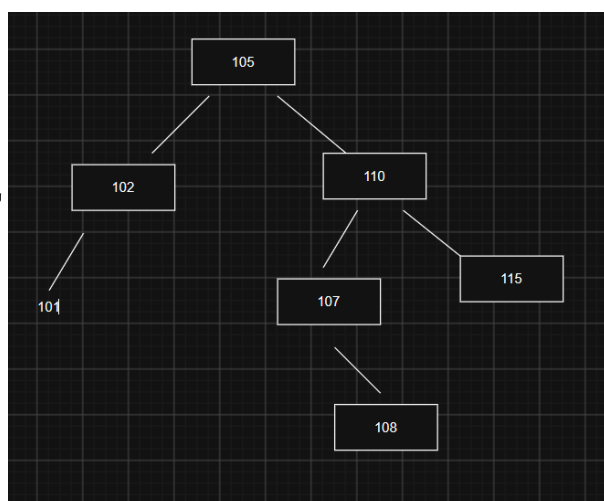
5



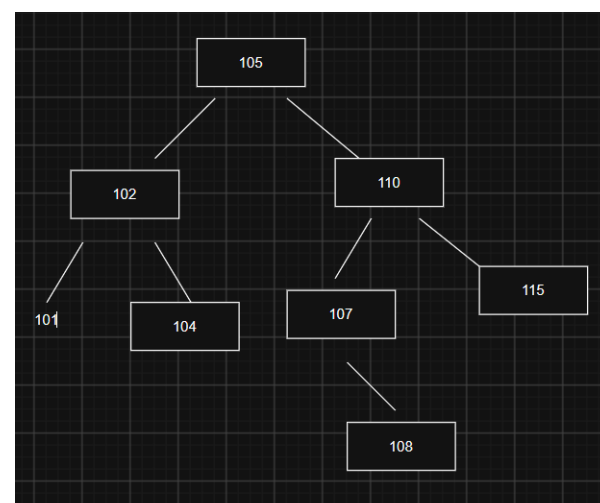
6



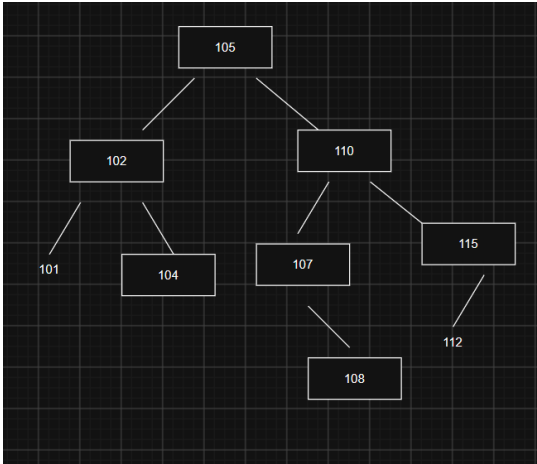
7



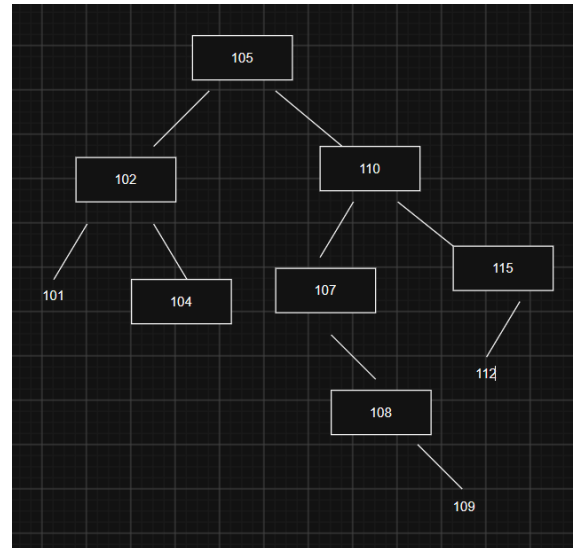
8



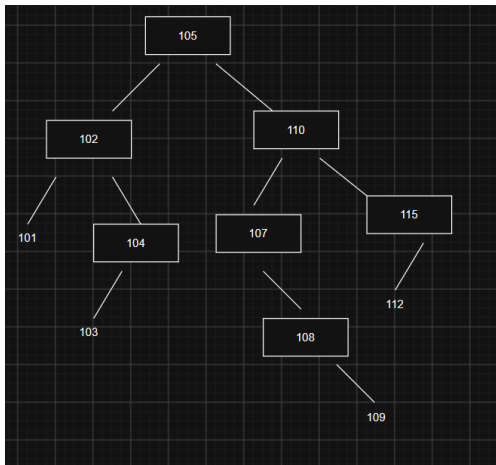
9



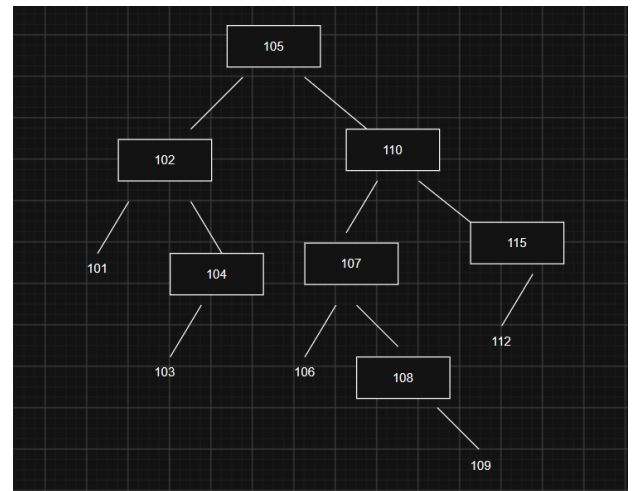
10



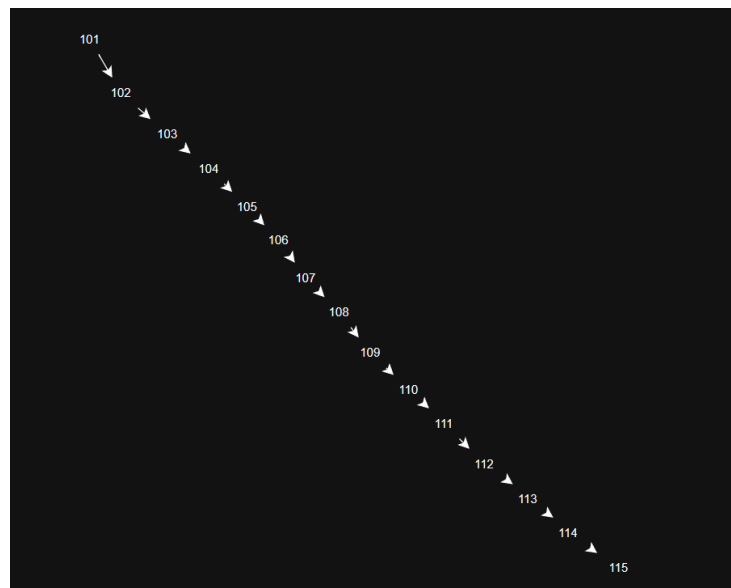
11



12



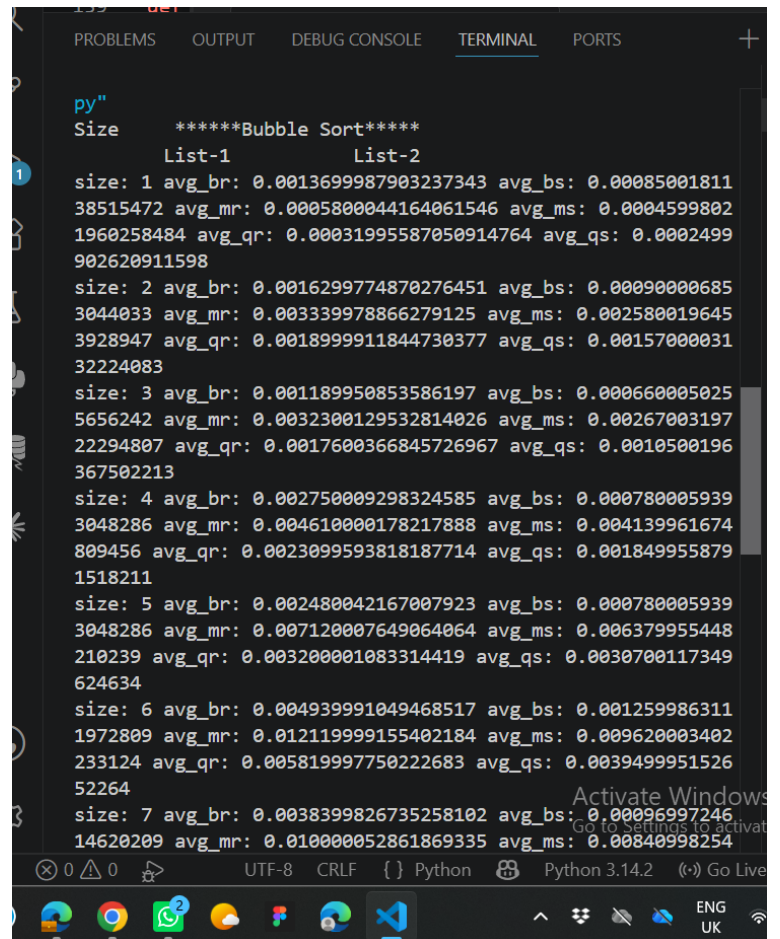
A3.2 = sorted insertion order



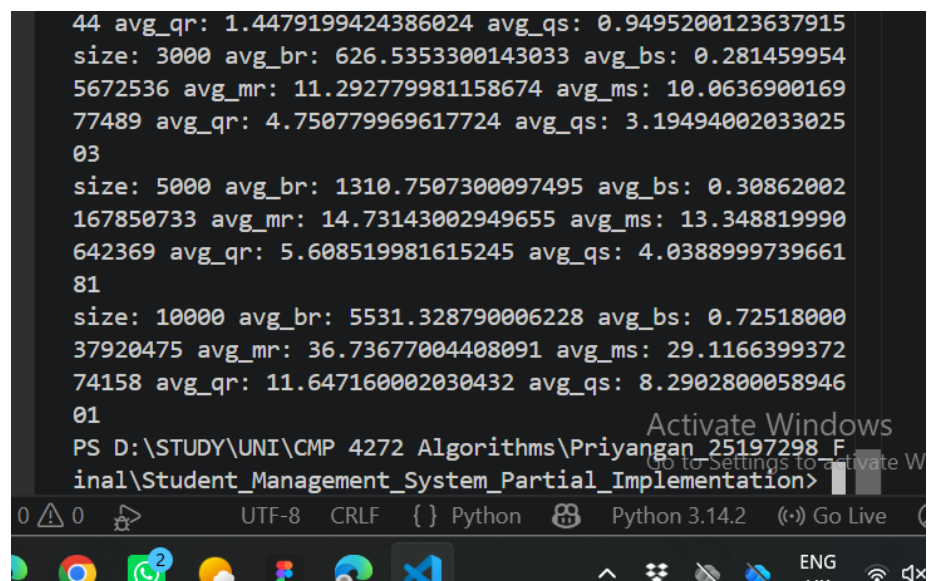
B1.1 – Average Execution Times

Bubble Sort, Merge Sort and Quick Sort were tested using two different types of input lists. List-1 represents a randomly generated list and List-2 represents a pre-sorted list. Different list sizes were tested and the average execution time was recorded in milliseconds.

Evidence of Termminal:



```
py"
Size      *****Bubble Sort*****
List-1    List-2
size: 1 avg_br: 0.0013699987903237343 avg_bs: 0.00085001811
38515472 avg_mr: 0.0005800044164061546 avg_ms: 0.0004599802
1960258484 avg_qr: 0.00031995587050914764 avg_qs: 0.0002499
902620911598
size: 2 avg_br: 0.0016299774870276451 avg_bs: 0.00090000685
3044033 avg_mr: 0.003339978866279125 avg_ms: 0.002580019645
3928947 avg_qr: 0.0018999911844730377 avg_qs: 0.00157000031
32224083
size: 3 avg_br: 0.001189950853586197 avg_bs: 0.000660005025
5656242 avg_mr: 0.0032300129532814026 avg_ms: 0.00267003197
22294807 avg_qr: 0.0017600366845726967 avg_qs: 0.0010500196
367502213
size: 4 avg_br: 0.002750009298324585 avg_bs: 0.000780005939
3048286 avg_mr: 0.004610000178217888 avg_ms: 0.004139961674
809456 avg_qr: 0.0023099593818187714 avg_qs: 0.001849955879
1518211
size: 5 avg_br: 0.002480042167007923 avg_bs: 0.000780005939
3048286 avg_mr: 0.007120007649064064 avg_ms: 0.006379955448
210239 avg_qr: 0.003200001083314419 avg_qs: 0.0030700117349
624634
size: 6 avg_br: 0.004939991049468517 avg_bs: 0.001259986311
1972809 avg_mr: 0.012119999155402184 avg_ms: 0.009620003402
233124 avg_qr: 0.005819997750222683 avg_qs: 0.0039499951526
52264
size: 7 avg_br: 0.0038399826735258102 avg_bs: 0.00096997246
14620209 avg_mr: 0.010000052861869335 avg_ms: 0.00840998254
```



```
44 avg_qr: 1.4479199424386024 avg_qs: 0.9495200123637915
size: 3000 avg_br: 626.5353300143033 avg_bs: 0.281459954
5672536 avg_mr: 11.292779981158674 avg_ms: 10.0636900169
77489 avg_qr: 4.750779969617724 avg_qs: 3.19494002033025
03
size: 5000 avg_br: 1310.7507300097495 avg_bs: 0.30862002
167850733 avg_mr: 14.73143002949655 avg_ms: 13.348819990
642369 avg_qr: 5.608519981615245 avg_qs: 4.0388999739661
81
size: 10000 avg_br: 5531.328790006228 avg_bs: 0.72518000
37920475 avg_mr: 36.73677004408091 avg_ms: 29.1166399372
74158 avg_qr: 11.647160002030432 avg_qs: 8.2902800058946
01
PS D:\STUDY\UNI\CMP 4272 Algorithms\Priyangan 25197298 F
inal\Student_Management_System_Partial_Implementation>
```

Table-1: Average execution times of Sorting methods

Size	Bubble Sort (List-1)	Bubble Sort (List-2)	Merge Sort (List-1)	Merge Sort (List2)	Quick Sort (List-1)	Quick Sort (List2)
1	0.001	0.001	0.000	0.000	0.000	0.000
2	0.001	0.001	0.003	0.002	0.002	0.001
3	0.002	0.001	0.004	0.004	0.003	0.001
4	0.002	0.001	0.005	0.004	0.003	0.002
5	0.003	0.001	0.007	0.006	0.003	0.003
10	0.007	0.001	0.016	0.013	0.009	0.006
100	0.580	0.009	0.260	0.225	0.106	0.077
1000	64.041	0.087	3.302	3.079	1.448	0.950
3000	626.535	0.281	11.293	10.064	4.751	3.195
5000	1310.751	0.309	14.731	13.349	5.609	4.039
10000	5531.329	0.725	36.737	29.117	11.647	8.290

B1.2:Terminal Evidence:

```

size: 10 avg_br: 0.004400010220706463 avg_bs: 0.00072007
53316283226 avg_mr: 0.010610022582113743 avg_ms: 0.00886
0020898282528 avg_qr: 0.004940037615597248 avg_qs: 0.003
940006718039513 avg_q1r: 0.004009995609521866 avg_q1s: 0
.0039800070226192474 avg_q2r: 0.004859967157244682 avg_q
2s: 0.006780005060136318
size: 100 avg_br: 0.348250032402575 avg_bs: 0.0046599423
51281643 avg_mr: 0.16564999241381884 avg_ms: 0.135680008
67962837 avg_qr: 0.06833004299551249 avg_qs: 0.046959985
04757881 avg_q1r: 0.05992997903376818 avg_q1s: 0.0495400
5125910044 avg_q2r: 0.06810000631958246 avg_q2s: 0.40011
995006352663
size: 1000 avg_br: 38.534580008126795 avg_bs: 0.05267004
016786814 avg_mr: 1.9633699674159288 avg_ms: 1.661109994
1655993 avg_qr: 0.8179499302059412 avg_qs: 0.56255001109
09343 avg_q1r: 0.7874099770560861 avg_q1s: 0.57939002290
36808 avg_q2r: 0.9843699168413877 avg_q2s: 36.3572000060
2305
size: 3000 avg_br: 356.1425100080669 avg_bs: 0.159309967
42099524 avg_mr: 6.699520023539662 avg_ms: 5.77225999440
9978 avg_qr: 2.7100799838081002 avg_qs: 1.94157005753368
14 avg_q1r: 2.725790021941066 avg_q1s: 1.934529980644583
7 avg_q2r: 3.1195199815556407 avg_q2s: 328.6399099742993
7
  
```

Table-2: Average execution times of Sorting methods

Size	Bubble Sort (List-1)	Bubble Sort (List-2)	Merge Sort (List-1)	Merge Sort (List2)	Quick Sort (List-1)	Quick Sort (List2)	Quick Sort_v1 (List-1)	Quick Sort_v1 (List2)	Quick Sort_v2 (List-1)	Quick Sort_v2 (List2)
1	0.001	0.001	0.000	0.000	0.000	0.000	0.0002	0.0001	0.0006	0.00052
2	0.001	0.001	0.003	0.002	0.002	0.001	0.0007	0.0006	0.0013	0.00095
3	0.002	0.001	0.004	0.004	0.003	0.001	0.001	0.0006	0.0017	0.0014
4	0.002	0.001	0.005	0.004	0.003	0.002	0.001	0.001	0.0021	0.0019
5	0.003	0.001	0.007	0.006	0.003	0.003	0.001	0.001	0.0024	0.0025
10	0.007	0.001	0.016	0.013	0.009	0.006	0.005	0.004	0.0069	0.0101
100	0.580	0.009	0.260	0.225	0.106	0.077	0.06	0.050	0.0607	0.3744
1000	64.041	0.087	3.302	3.079	1.448	0.950	0.8	0.565	0.8753	36.9413
3000	626.535	0.281	11.293	10.064	4.751	3.195	2.6	2.024	3.1280	323.70
5000	1310.751	0.309	14.731	13.349	5.609	4.039	4.5	3.232	5.3923	915.30
10000	5531.329	0.725	36.737	29.117	11.647	8.290	9.6	6.851	11.9435	3783.96

B1.3 – Performance Comparison of Sorting Algorithms

The results in Table-2 show that the performance of the sorting algorithms changes depending on the size and arrangement of the input data. Bubble Sort, Merge Sort, Quick Sort and the two modified versions of Quick Sort were compared using random and pre-sorted lists.

Bubble Sort performed well for very small lists, but its execution time increased significantly when the size of the random list increased. For example, with 10,000 random elements, Bubble Sort took around 4120.91 ms. However, its performance was much faster with the pre-sorted list. This shows that Bubble Sort becomes inefficient for large random datasets.

Merge Sort showed more consistent performance for both random and pre-sorted lists. At 10,000 elements, it took around 25.92 ms for the random list and 22.59 ms for the pre-sorted list. Compared with Bubble Sort, the increase in execution time was much smaller as the input size increased.

Quick Sort Version 2 showed a clear difference between random and pre-sorted input. For 10,000 random elements, it took around 11.94 ms, but for the pre-sorted list it increased significantly to around 3783.96 ms. This shows that the performance of Quick Sort can be strongly affected by the way the pivot is selected and the arrangement of the input data.

Theoretical and Practical Performance

The theoretical time complexity helps to explain the results from the practical testing. Bubble Sort can have a time complexity of $O(n^2)$, which explains why its execution time increased significantly for large random lists. Merge Sort has a time complexity of $O(n \log n)$, so it showed more stable performance when the input size increased.

Quick Sort normally performs efficiently when the pivot divides the data into balanced parts. However, its performance can become $O(n^2)$ when the partitions are highly unbalanced. This can be seen clearly in Quick Sort Version 2, where the execution time increased significantly for the pre-sorted list.

From these results, there is no single sorting method that performs the same for every type of input. The experiment shows that the choice of algorithm, pivot selection and input arrangement can have a significant effect on execution time. For larger datasets in this experiment, Merge Sort, the original Quick Sort and Quick Sort Version 1 were more efficient than Bubble Sort, while Quick Sort Version 2 was strongly affected by pre-sorted input.

B2 – Graph Representation and Performance Analysis

The student friendship system uses a graph to store students and their friendships. Each student has a list of friends. This representation is suitable because it only stores the friendships that actually exist.

Adding a new friendship is simple because the student can be added to the other student's friend list. Checking whether two students are friends requires searching the friend list.

Another option is an adjacency matrix. It can check a friendship very quickly, but it uses more memory because space is needed for every possible connection. Therefore, the current list-based representation is more suitable when students have only a few friendships. An adjacency matrix would be more suitable for a small graph with many friendships.

B3 – Analysis of Searching Algorithms

The `search()` method searches for a student using the student ID. It compares the ID with the current node. If the ID is smaller, it moves to the left side of the tree. If the ID is larger, it moves to the right side. Therefore, it can find a student without checking every record.

The `locate_student()` method works differently. It searches through the tree using recursion. It checks the current node first, then the left side and finally the right side. Because of this, it may need to check many students before finding the correct student.

When searching for student IDs **101 and 115**, the `search()` method follows the correct path based on the ID values. The `locate_student()` method may visit more nodes because it does not use the ordering of the BST.

For a balanced tree, the average time complexity of `search()` is **$O(\log n)$** . In the worst case, it can become **$O(n)$** . The time complexity of `locate_student()` is **$O(n)$** because it may need to check all the nodes. Therefore, the `search()` method is generally more efficient for finding a student by ID.